Arduino IDE

Presentation

By

Dr. Kesavan Gopal

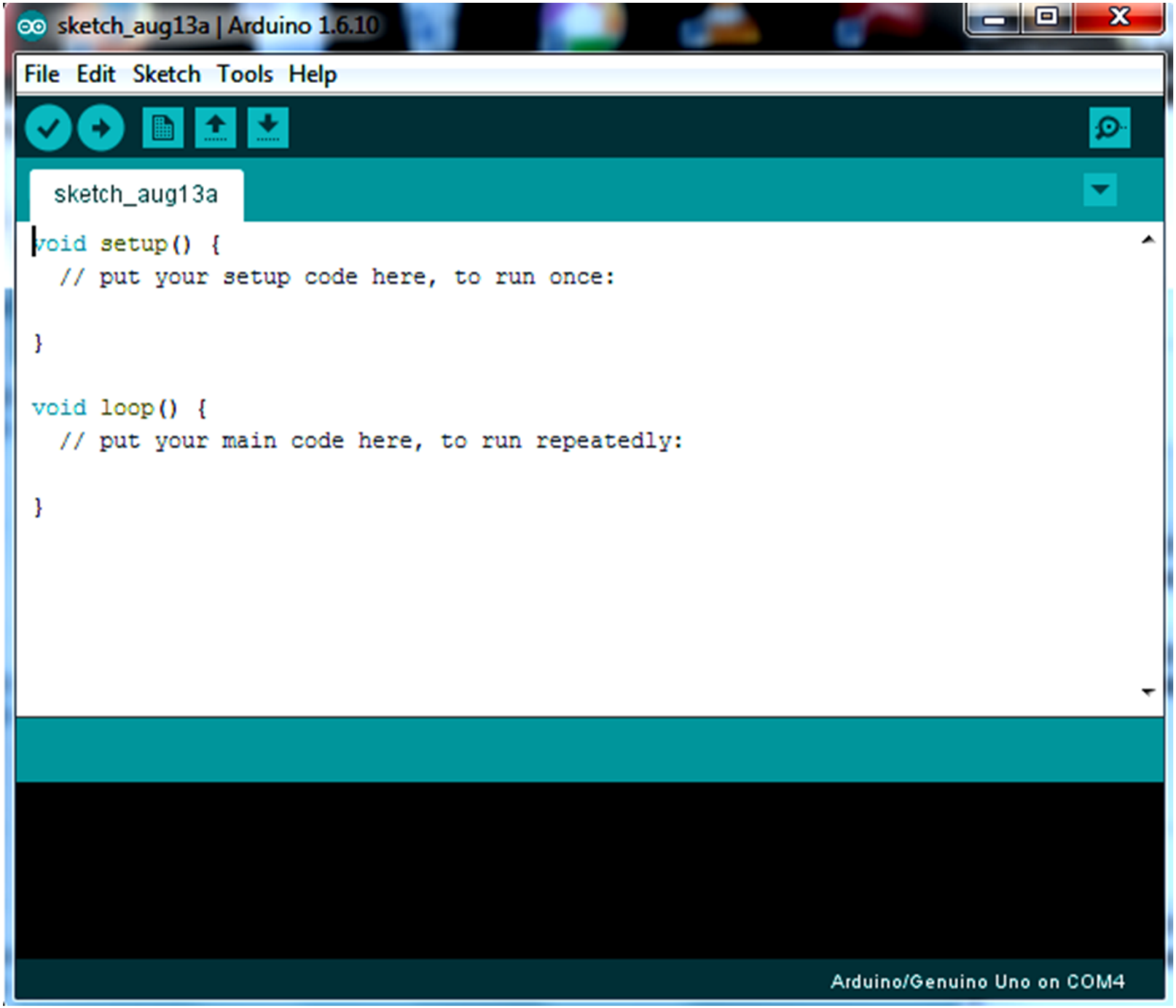
Outline

- Installing Arduino IDE
- Setting up Arduino board,
- Preparing IDE & Arduino sketch, Saving
- Uploading and running the sketch

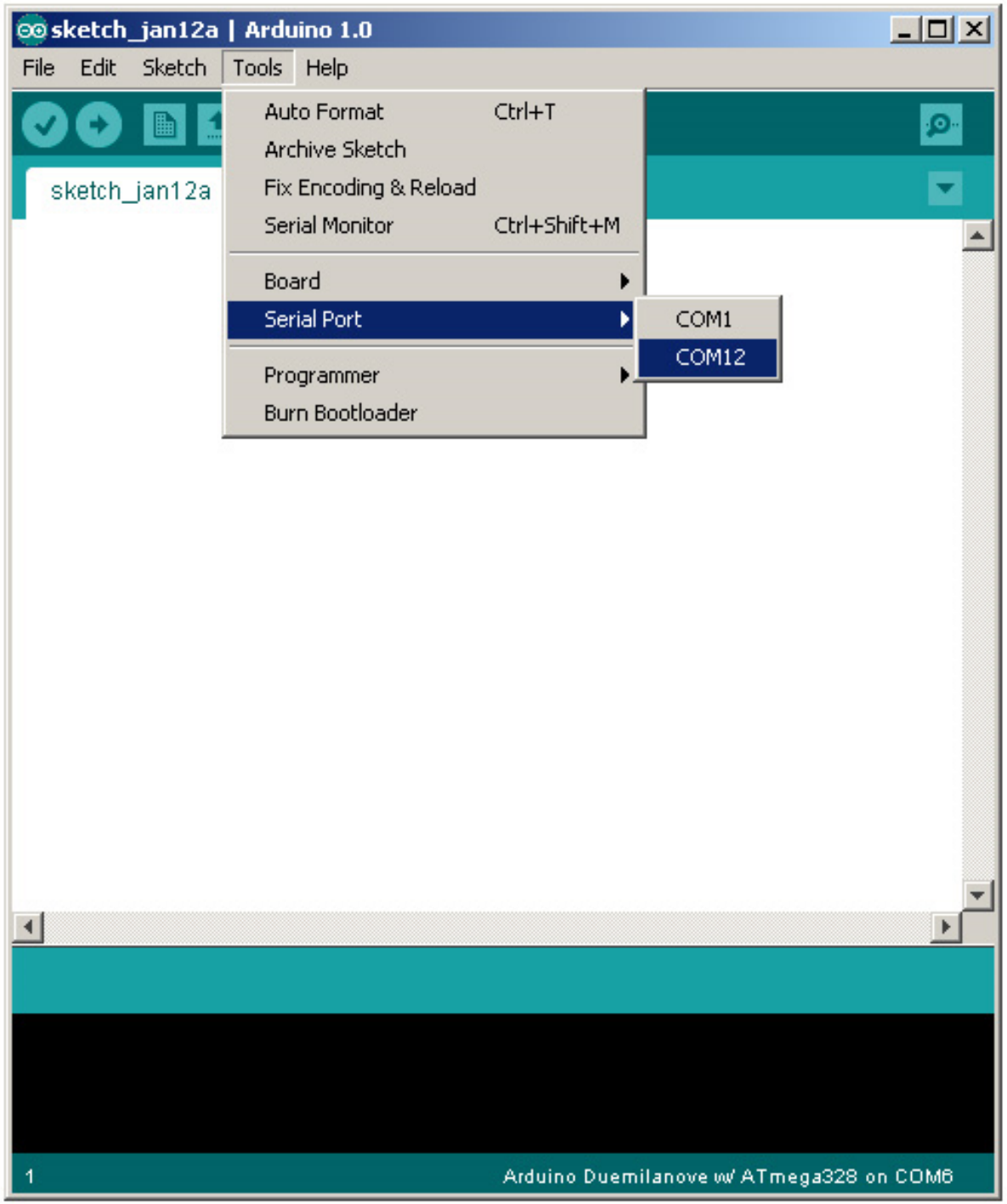
Installing Arduino IDE

- Download & install the Arduino environment (IDE-Integrated Development Environment)
- http://arduino.cc/en/Guide/HomePage
- Connect the board to your computer via the USB cable
- If needed, install the drivers
- Launch the Arduino IDE Select your serial port
- Select your board

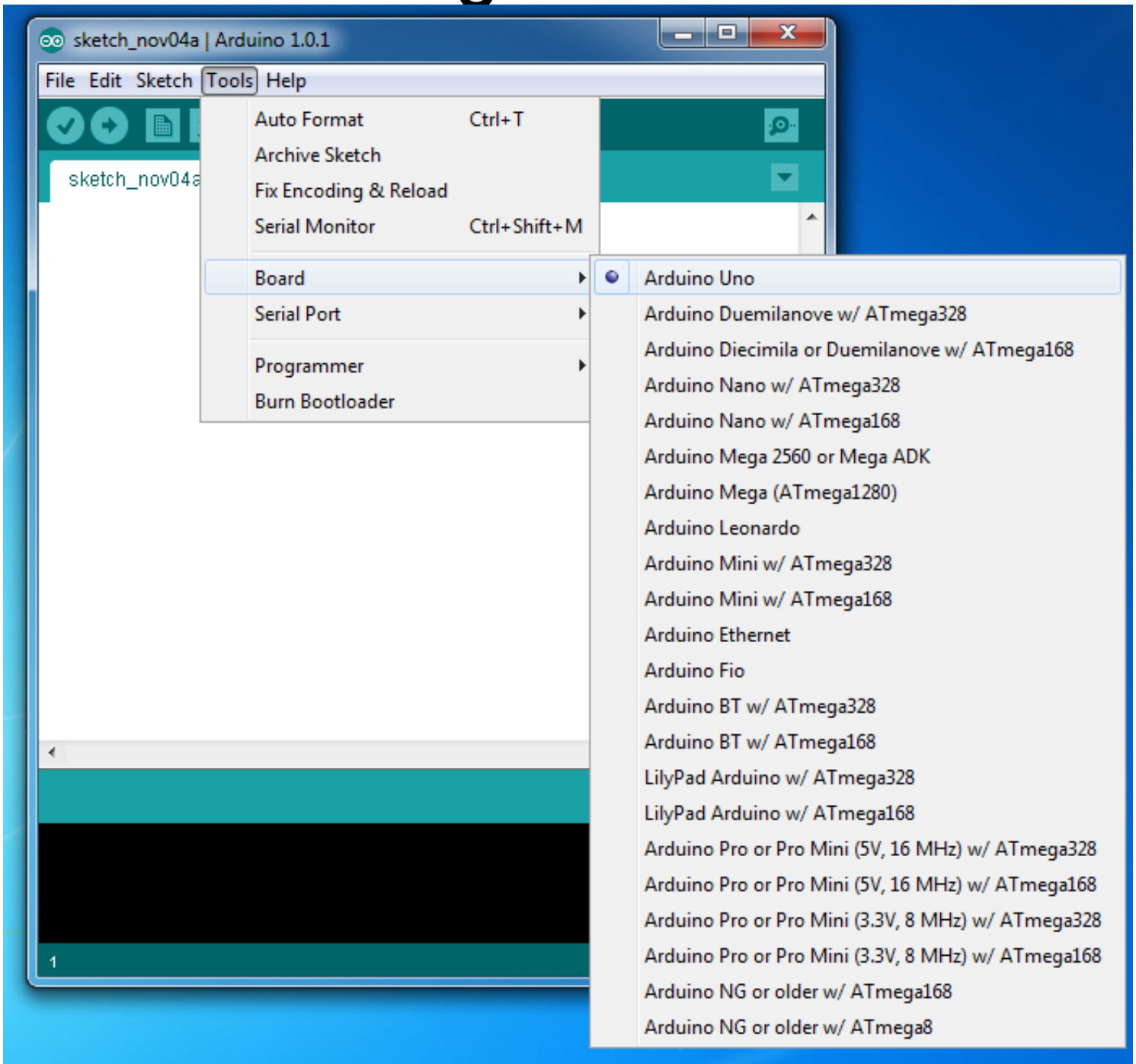
Cont...


sketch_aug13a | Arduino 1.6.10
File Edit Sketch Tools Help
sketch_aug13a
void setup() {
// put your setup code here, to run once:
}
void loop() {
// put your main code here, to run repeatedly:
}
Arduino/Genuino Uno on COM4

Selecting Serial Port

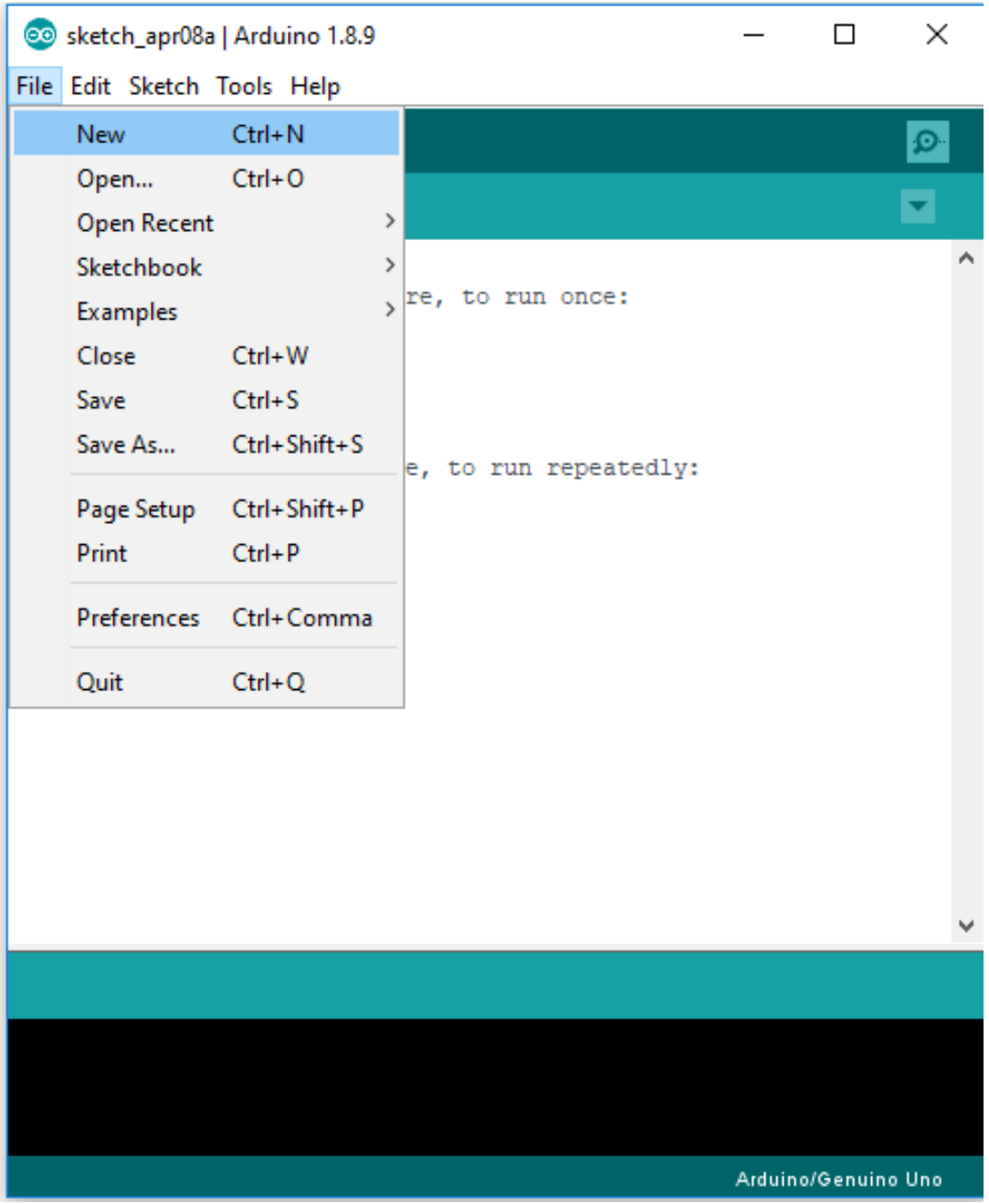


Selecting the Board



Arduino Development Flow

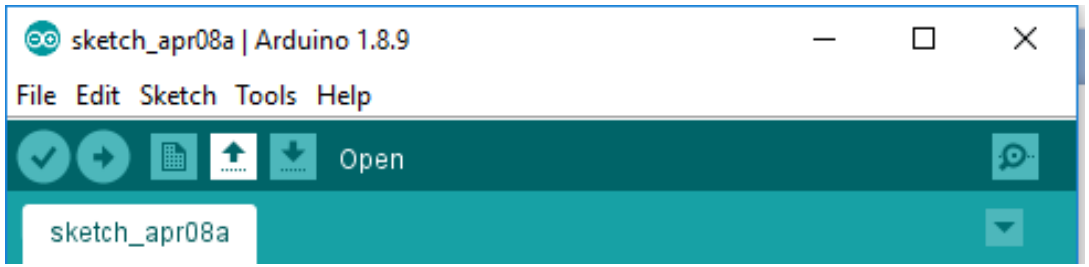
Step 1: New Sketch



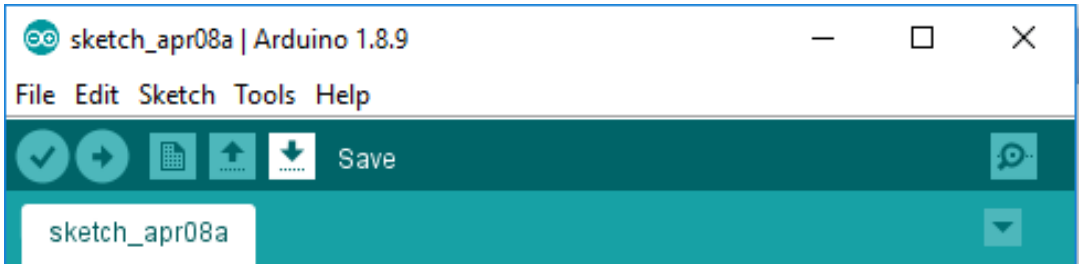
Cont...

- New Sketch

- Opening an existing sketch

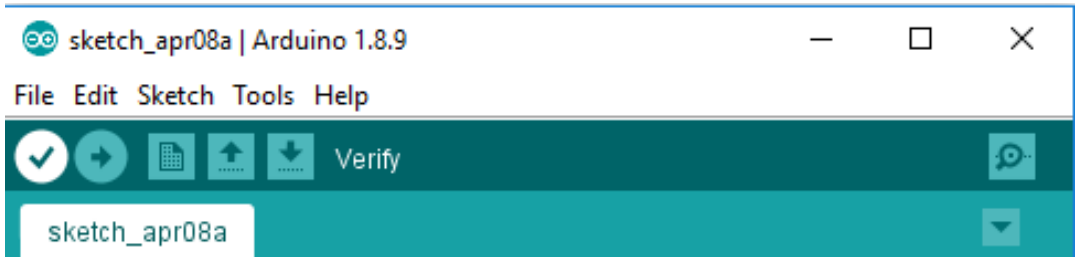


- Save a sketch

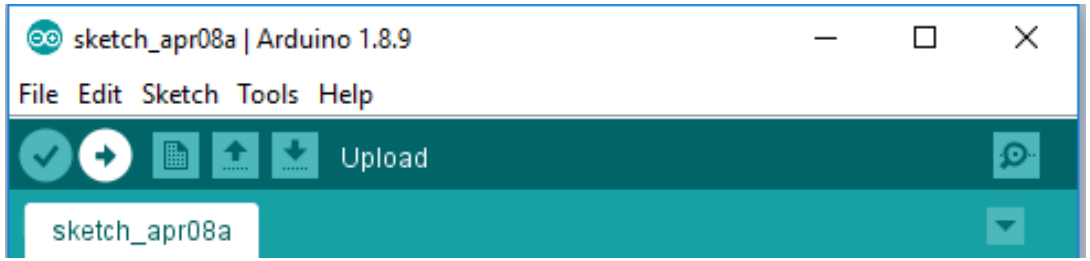


Cont...

- Verify the Sketch



- Upload the Sketch



Summarily

- Step 1: Write your Sketch
- Step 2: Verify the Sketch
- Step 3: Upload the sketch to Arduino Board.

Arduino Coding Basics

- TWO Mandatory functions in Arduino Coding

```
void setup() {
// Only execute once when the Arduino boots
}

void loop(){
// Code executes top-down and repeats
   continuously
}
```

Cont...

- Setup()
  - Executes once during **power up** or **Reset**
  - Fn used to **configure** the Arduino **Pins**
- Loop ()
  - Executes Continuously
  - Top-down
  - Upon end starts from Top

Simple Example

```
// Global Variables
int var1 = 10;
int var2 = 20;
void setup() {
// Only execute once when the Arduino boots
var2 = 5; // var2 becomes 5 once the Arduino boots
}
void loop(){
// Code executes top-down and repeats continuously
if (var1 > var2){ // If var1 is greater than var2
var2++; // Increment var2 by 1
} else { // If var1 is NOT greater than var2
var2 = 0; // var2 becomes 0
} }
```

Arduino Pins

- Main feature of Arduino is control over digital Input/output(I/O) Pins.
- Writing to Pin
  - Logic HIGH – applying a Voltage of 5V to the pins
  - Logic LOW - applying a Voltage of 0V to the pins
- Reading from Pin
  - Can read from Pins, 5 V interpreted as - Logic HIGH
  - 0 V interpreted as – Logic LOW

Arduino Pins- Example

```
void setup() {
pinMode(2, OUTPUT);      // Set pin 2 as a digital Output
pinMode(3, INPUT);       // Set pin 3 as a digital Input
}
void loop(){
digitalWrite(2, HIGH);   // Set pin 2 HIGH
delay(100);              // Wait 100 milliseconds
digitalWrite(2, LOW);    // Set pin 2 LOW
delay(100);              // Wait 100 milliseconds
int pinValue = digitalRead(3);
// Read the value of pin 3 and store it in a variable
}
```

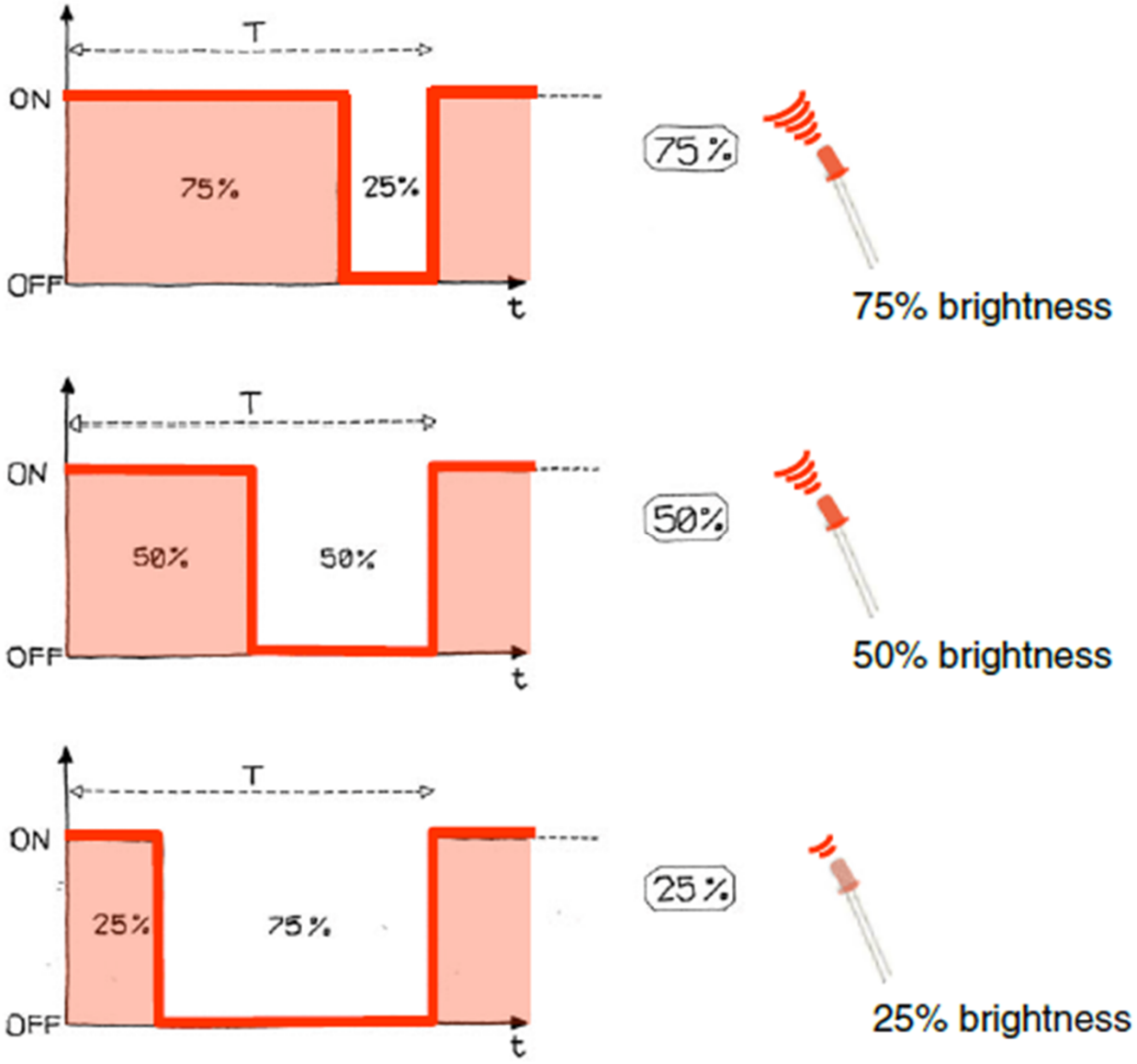
Blinking LED - Example

Arduino Supports - PWM

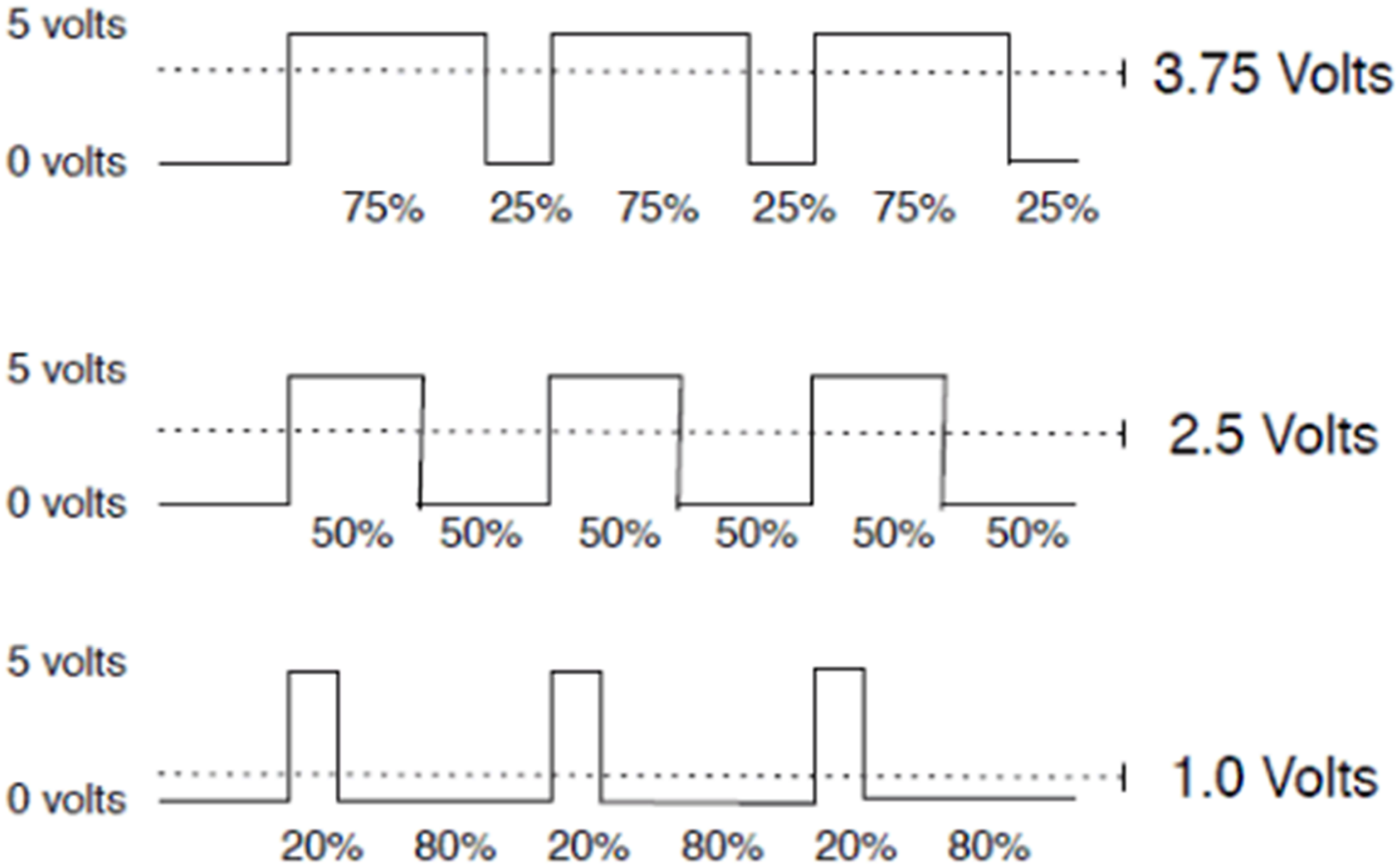
PWM – Pulse Width Modulation

Dedicated PWM pins – Arduino Uno – 3,5,6,9,10,11

- Can’t use digital pins to directly supply say 2.5V, but can pulse the output on and off really fast to produce the same effect
- The on-off pulsing happens so quickly, the connected output device “sees” the result as a reduction in the voltage



PWM – Duty Cycle



PWM Pins – Analog Fade Value

- Command: **analogWrite(pin,value)**
- **value** is duty cycle: between 0 and 255

Examples:

```
analogWrite(9, 128)  //   for a 50% duty cycle
analogWrite(11, 64) //   for a 25% duty cycle
```

Fade LED - Example

```
int ledPin = 9;
void setup()  {}
void loop()  {
for (int fadeValue = 0 ; fadeValue <= 255; fadeValue += 5) {
analogWrite(ledPin, fadeValue); delay(30);  }

for (int fadeValue = 255 ; fadeValue >= 0; fadeValue -
    = 5) {
analogWrite(ledPin, fadeValue);delay(30);
} }
```

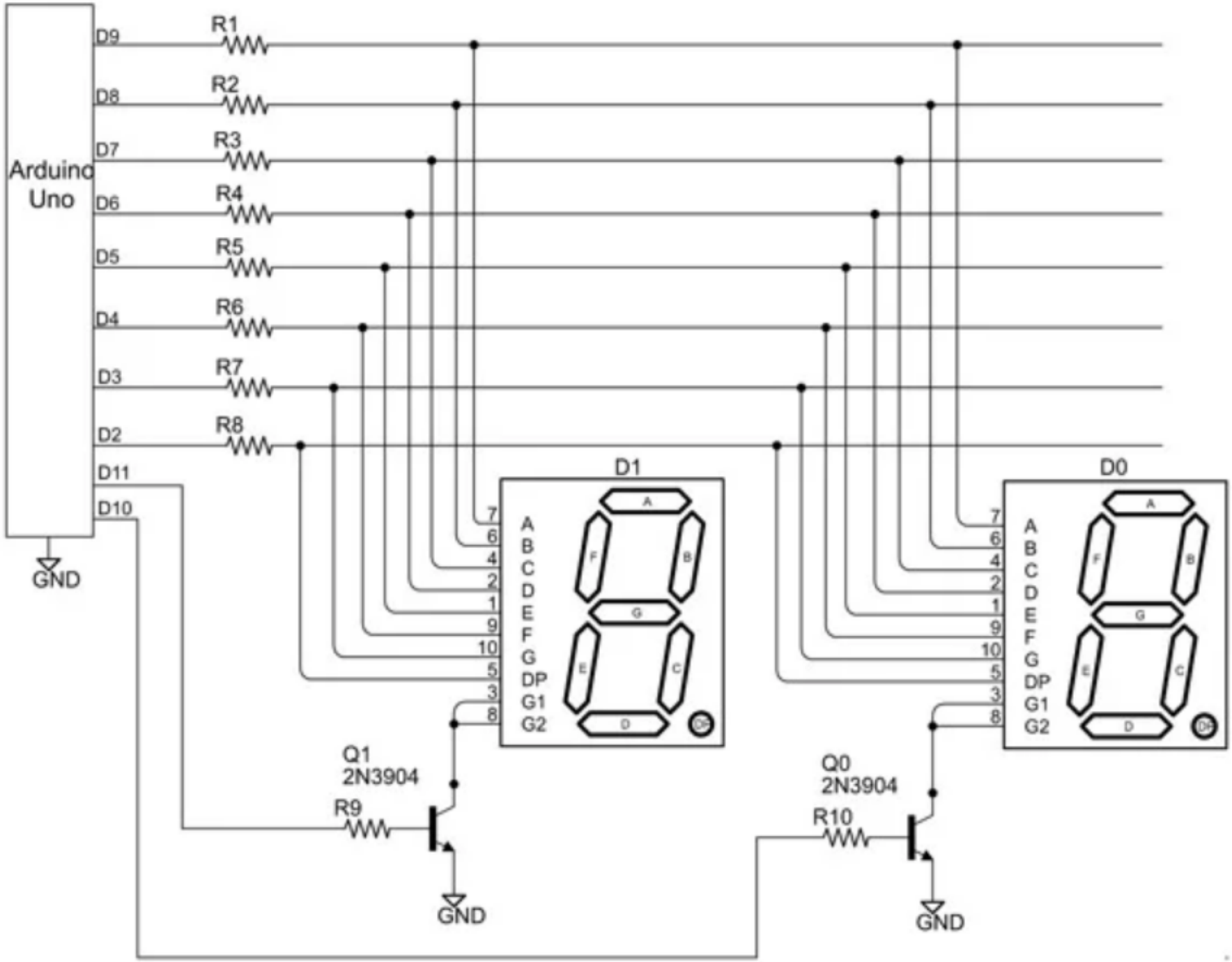
Connecting a 7 –Segment display

Change over to alternate document for better readability

Multi digit Seven Segment

- 2 or more seven segment displays
- Two additional pins are required to control the ON and OFF of each segment
- To send sequence of numbers like 00 to 99

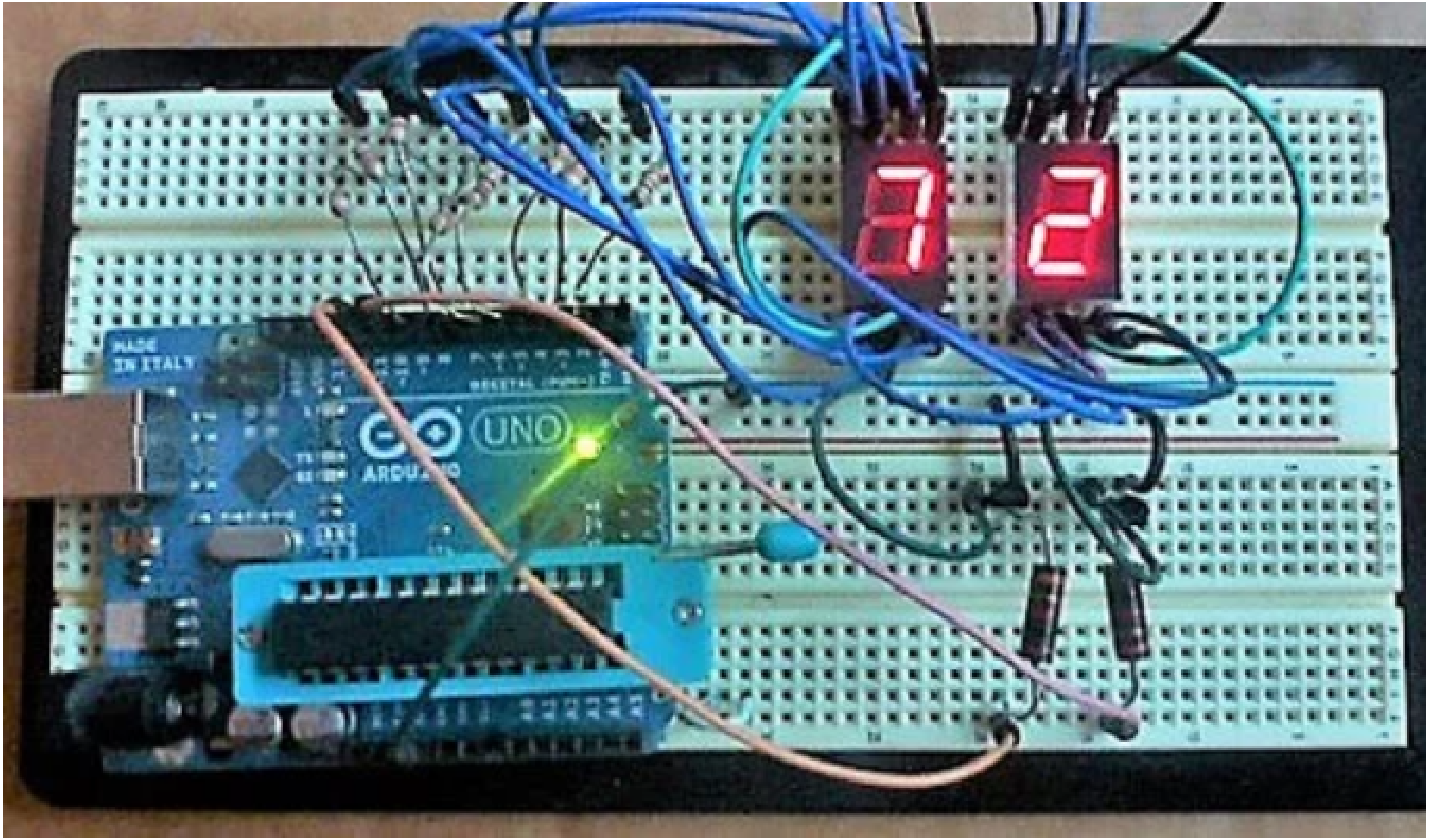
Circuit


Arduino Uno
D9
D8
D7
D6
D5
D4
D3
D2
D11
D10
GND
R1
R2
R3
R4
R5
R6
R7
R8
D1
D0
A
B
C
D
E
F
G
DP
G1
G2
Q1
2N3904
R9
Q0
2N3904
R10
GND
GND

Arduino Sketch

- Change over to Alternate document for better readability.

Multi Segment Implementation



LCD Interfacing

- Interfacing requirement:
  - RS - Register Select
  - RW - Read / Write
  - E - Enable
- Command Write (RS -0 ; RW – 0; E – 0, D7-D0 -Cmd)
- Data Write (RS-1, RW-0;E-1; D7-D0 – Data Write)
- BS

LCD interface – Arduino Sketch

```
#include<LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2); RS,E,D4,D5,D6,D7
void setup()
{
lcd.begin(16, 2);
}
void loop() {
lcd.setCursor(0,0);
lcd.print("16x2 LCD MODULE");
lcd.setCursor(2,1);
lcd.print("HELLO WORLD");
 }
```

LCD interface – Arduino functions

- LiquidCrystal()
- LiquidCrystal(rs, enable, d4, d5, d6, d7)
  LiquidCrystal(rs, rw, enable, d4, d5, d6, d7)
  LiquidCrystal(rs, enable, d0, d1, d2, d3, d4, d5, d6, d7)
  LiquidCrystal(rs, rw, enable, d0, d1, d2, d3, d4, d5, d6, d7)
- begin() *lcd*.begin(cols, rows)
- clear() *lcd*.clear()
- setCursor() *lcd*.setCursor(col, row)
- write() *lcd*.write(data)
- print() *lcd*.print(data)
- cursor() *lcd*.cursor()
- noCursor() *lcd*.noCursor()
- blink() *lcd*.blink() // Work around

LCD interface – Arduino functions Cont..

- blink() *lcd*.blink()
- noBlink() *lcd*.noBlink()
- display() *lcd*.display()
- noDisplay() *lcd*.noDisplay()
- scrollDisplayLeft() *lcd*.scrollDisplayLeft()
- scrollDisplayRight() *lcd*.scrollDisplayRight()
- autoscroll() *lcd*.autoscroll()
- noAutoscroll() *lcd*.noAutoscroll()
- leftToRight() *lcd*.leftToRight()
- rightToLeft() *lcd*.rightToLeft()
- createChar() *lcd*.createChar(num, data)

General purpose input - switches

Switch types & states

- **Momentary and maintained switches**
  - **Momentary** - Active as long as they are pressed - key board
  - **Maintained** - keep the state we let them in - light switch
- Open or closed switches

  Momentary switch

  Closed Switch - conduct current while pressed

  Open switch - normal state - no current conduction
- poles and Throws
  - SPST – Switches have closed state when they conduct current
  - Open state - no current no conduction
  - SPDT - switch route the current from common pole to any one of the outputs

Arduino Uno – Reading a Switch

```
// Declare the pins for the Button and the LED
int buttonPin = 2;
int LED = 13;
void setup() {
// Define pin #2 as input
pinMode(buttonPin, INPUT);
// Define pin #13 as output, for the LED
pinMode(LED, OUTPUT);
}
```

Cont...

```
void loop() {
// Read the value of the input. It can either be 1 or 0.
int buttonValue = digitalRead(buttonPin);
if (buttonValue == HIGH) {
// If button pushed, turn LED on
digitalWrite(LED,HIGH);
} else {
// Otherwise, turn the LED off
digitalWrite(LED, LOW);
}
}
```

Issues around - Switch

- Floating values – Neither LOW Nor HIGH
- Just declare the global variable to have its initial floating status:
- int buttonValue = digitalRead (buttonPin);
- Pull-up Resistors
- Pull down Resistors – both Can be connected externally
- Inverted input logic possible through pull-up/pull-down

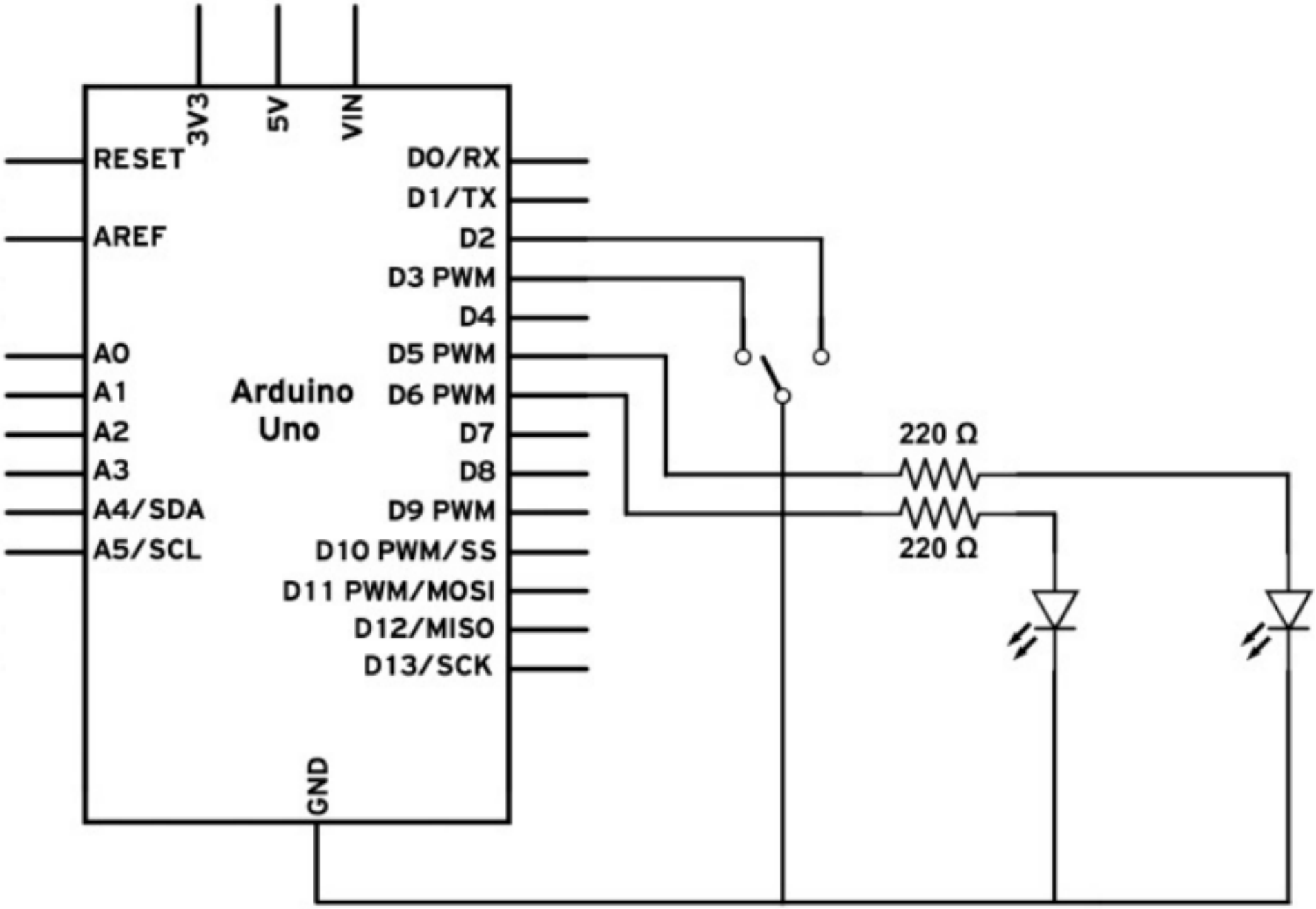
Arduino Uno – Internal Pull-up Resistor

```
// Declare the pins for the Button and the LED
int buttonPin = 12;
int LED = 13;
void setup() {
// Define pin #12 as input and activate the internal
   pull-up resistor
pinMode(buttonPin, INPUT_PULLUP);
// Define pin #13 as output, for the LED
pinMode(LED, OUTPUT);
}
```

Cont...

```
void loop(){
// Read the value of the input. It can either be 1 or 0
int buttonValue = digitalRead(buttonPin);
if (buttonValue == LOW){
// If button pushed, turn LED on
digitalWrite(LED,HIGH);
} else {
// Otherwise, turn the LED off
digitalWrite(LED, LOW);
}
}
```

SPDT Switch


3V3
5V
VIN
RESET
AREF
A0
A1
A2
A3
A4/SDA
A5/SCL
Arduino
Uno
D0/RX
D1/TX
D2
D3 PWM
D4
D5 PWM
D6 PWM
D7
D8
D9 PWM
D10 PWM/SS
D11 PWM/MOSI
D12/MISO
D13/SCK
GND
220 Ω
220 Ω

Cont...

```
int buttonPin1 = 2; int buttonPin2 = 3;
int LED1 = 5; int LED2 = 6;
void setup() {
// Define the two LED pins as outputs
pinMode(LED1, OUTPUT);
pinMode(LED2, OUTPUT);
// Define the two buttons as inputs with the internal pull-up
   resistor
activated
pinMode(buttonPin1, INPUT_PULLUP);
pinMode(buttonPin2, INPUT_PULLUP);
}
```

```
void loop(){
// Read the value of the inputs. It can be either 0 or 1
// 0 if toggled in that direction and 1 otherwise
int buttonValue1 = digitalRead(buttonPin1);
int buttonValue2 = digitalRead(buttonPin2);
if (buttonValue1 == HIGH && buttonValue2 == HIGH){
// Switch toggled to the middle. Turn LEDs off
digitalWrite(LED1, LOW);
digitalWrite(LED2, LOW);
} else if (buttonValue1 == LOW){
// Button is toggled to the second pin
 digitalWrite(LED1, LOW);
digitalWrite(LED2, HIGH);
} else {
// Button is toggled to the third pin
digitalWrite(LED1, HIGH);
digitalWrite(LED2, LOW);
}
}
```

Switch Status – Serial Connection

```
int buttonPin = 2;
void setup() {
// Define pin #2 as input
pinMode(buttonPin, INPUT_PULLUP);
// Establish the Serial connection with a
  //baud rate of 9600
Serial.begin(9600);
}
```

```
void loop(){
// Read the value of the input. It can either be 1
   or 0.
int buttonValue = digitalRead(buttonPin);
// Send the button value to the serial
   connection
Serial.println(buttonValue);
// Delays the execution to allow time for the
   serial transmission
delay(25);
}
```

Sensor Interfacing

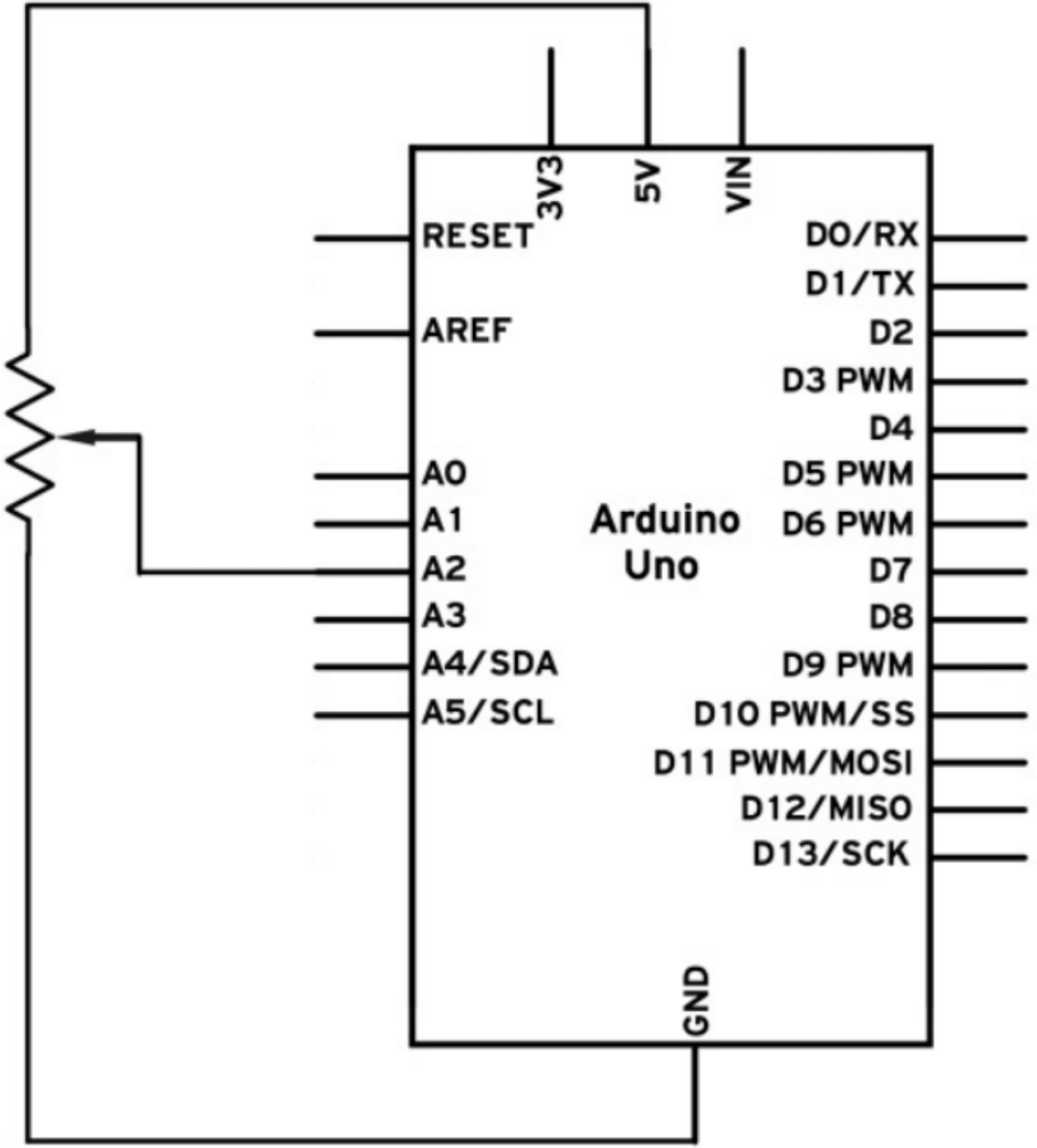
Acquiring data from the environment

Examples - Sound , Radiation, More sensors @ the end

Simple Sensor - Potentiometer

- Variable resistor , modify the internal resistance

Potentiometer Interface – Arduino Uno



Arduino Sketch

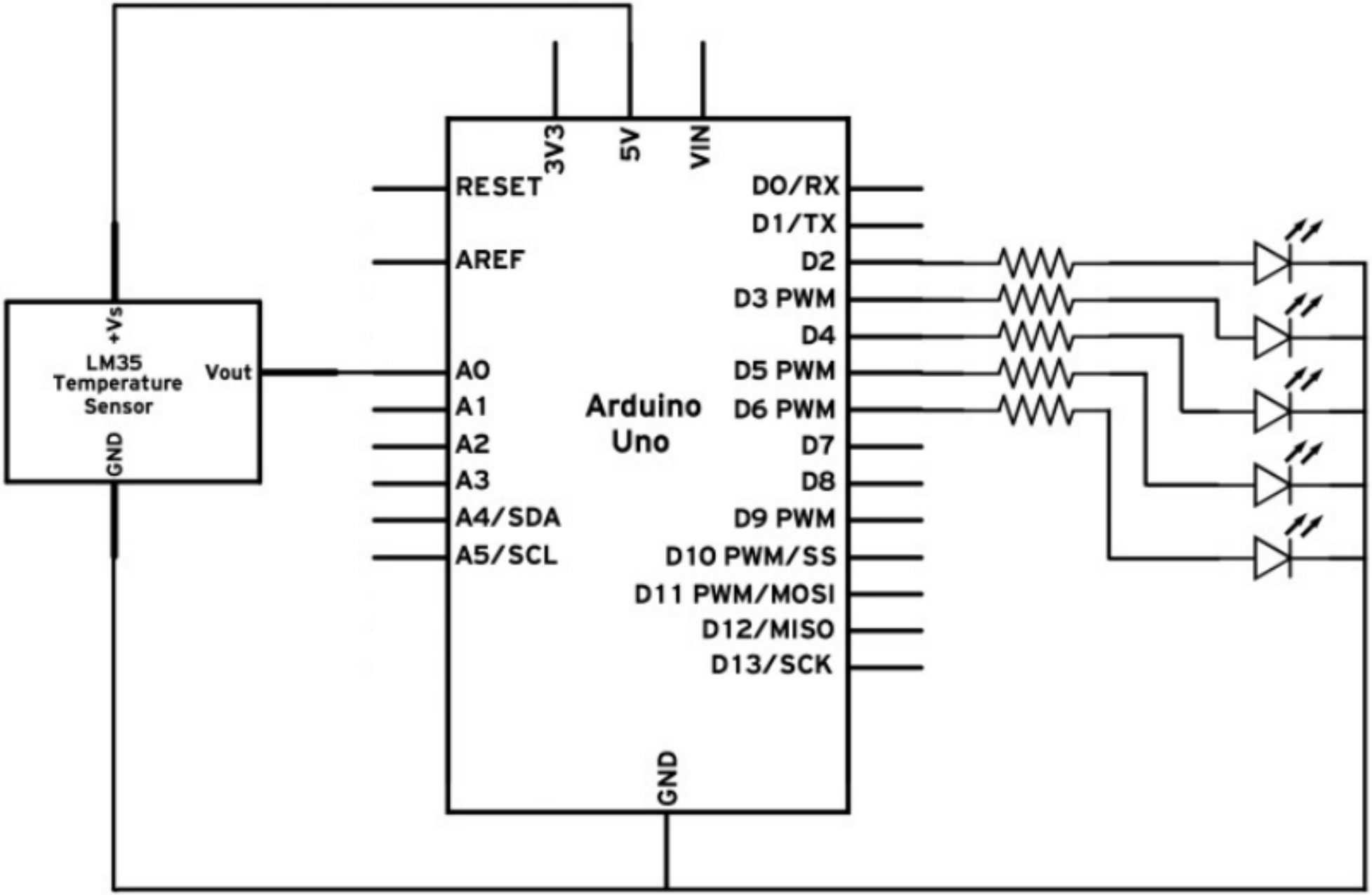
```
int LED = 13; // Declare the built-in LED
int sensorPin = A2; // Declare the analog port we connected
void setup(){
// Start the Serial connection
Serial.begin(9600);
// Set the built in LED pin as OUTPUT
pinMode(LED, OUTPUT);
}

// LED blinking rate control
// Intended to sense the voltage at the centre terminal of
// potentiometer
```

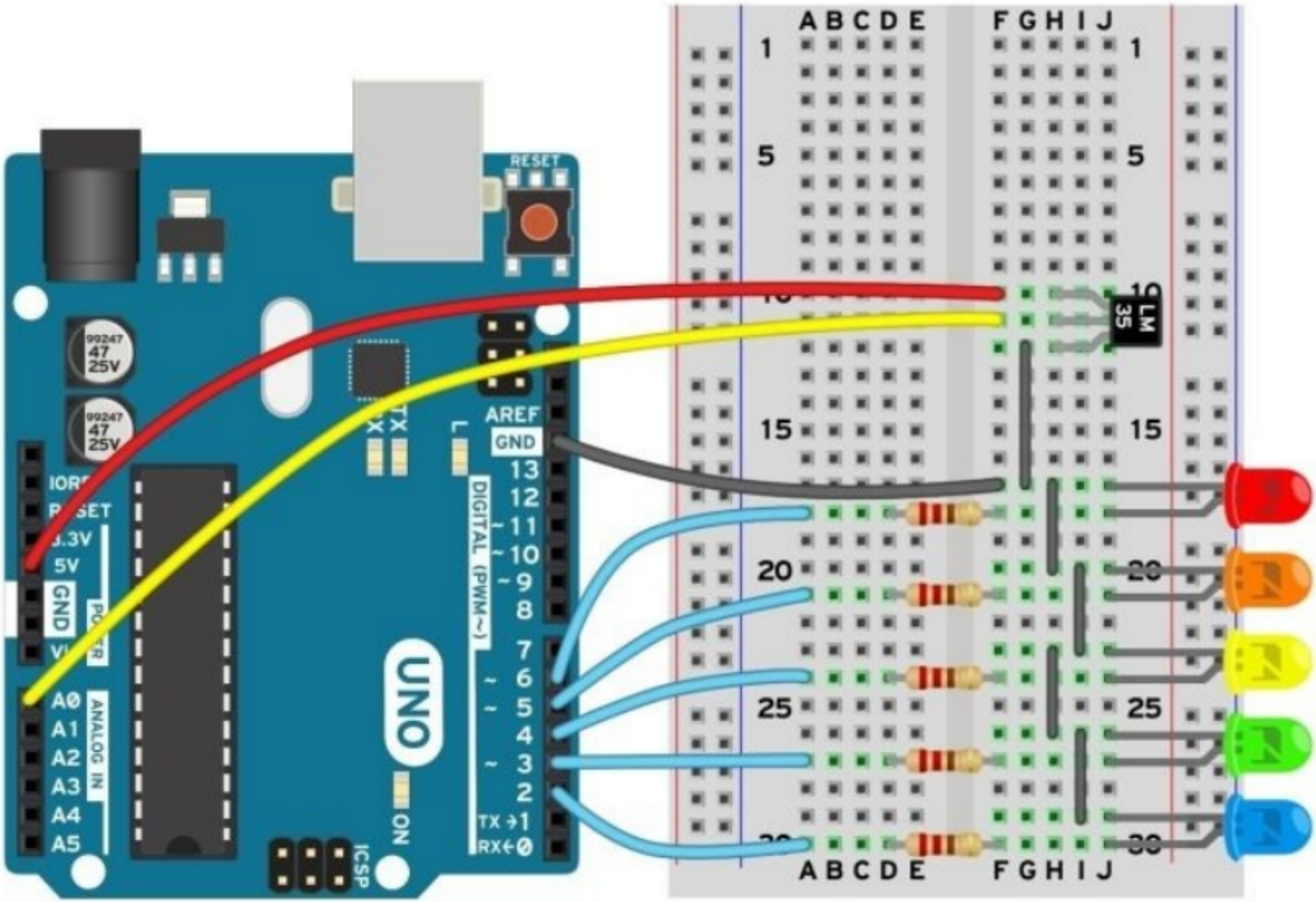
Cont...

```
void loop(){
// Read the value of the sensor
int val = analogRead(sensorPin);
// Print it to the Serial
Serial.println(val);
// Blink the LED with a delay of a forth of the sensor value
digitalWrite(LED, HIGH);
delay(val/4);
digitalWrite(LED, LOW);
delay(val/4);
}
```

Temperature Sensor Interface- Arduino Uno



Cont...


RESET
AREF
GND
DIGITAL (PWM~)
ANALOG IN
UNO
ON
ICSP
A B C D E
F G H I J
LM
35

Arduino Sketch

Change over to alternate document for better Readability.

Computations

LM35 - 10mV for each degree Celsius

5V returns – 1023 (Since ADC is in10-bit range)

25 degrees return – 250mV or 0.25 V – serial print yields 0.25 V/ 5V * 1023 = 51 (Approx.)

WOW – Range of Sensors

Temperature Distance Resistance

Compass Gas Sensor Humidity

Radiation Capacitance Infrared

Light Pressure Interference

Acceleration Sound Altitude Current

Orientation Weight Pulse Depth

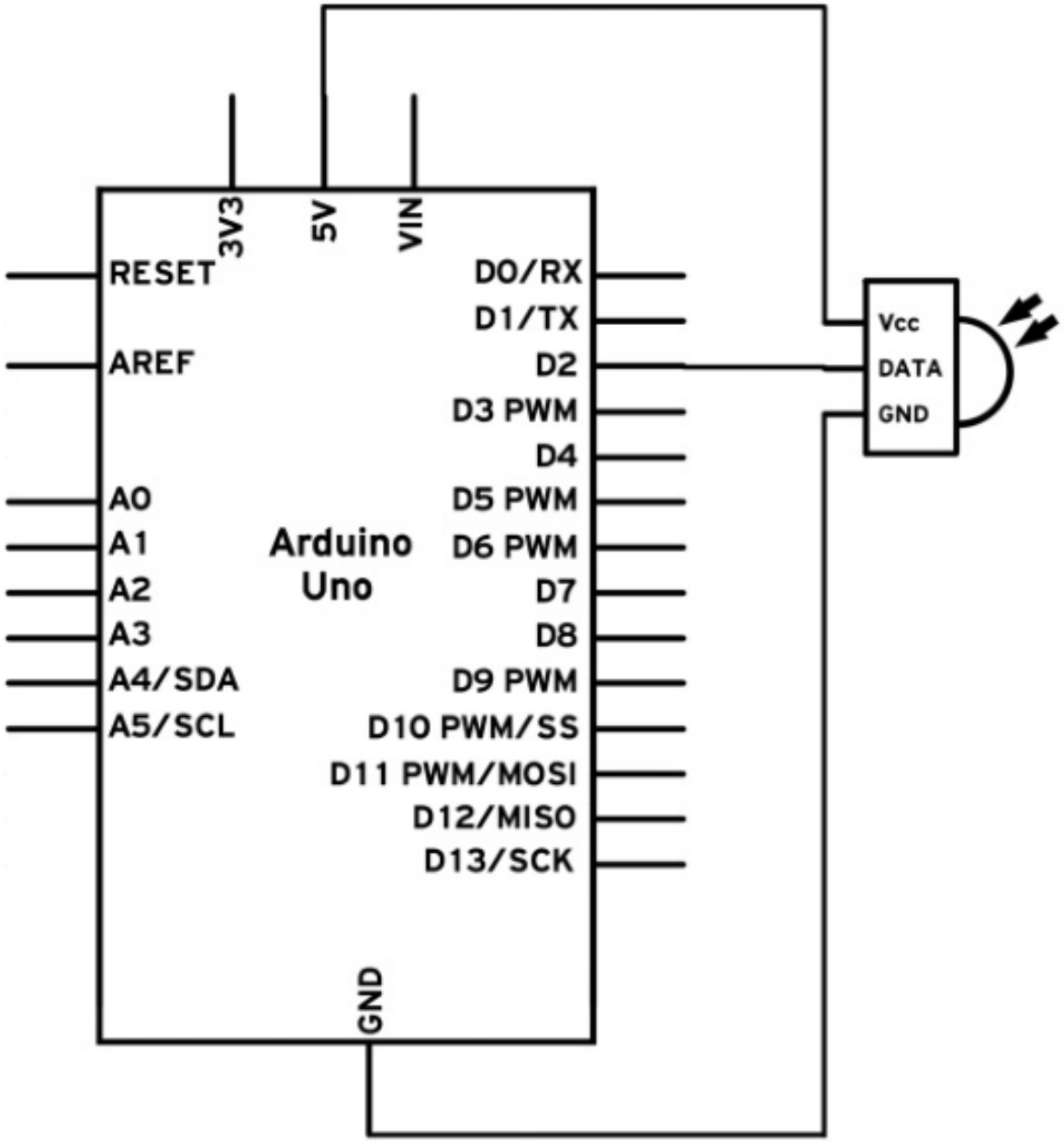
Voltage Angular Velocity Force

Fingerprint

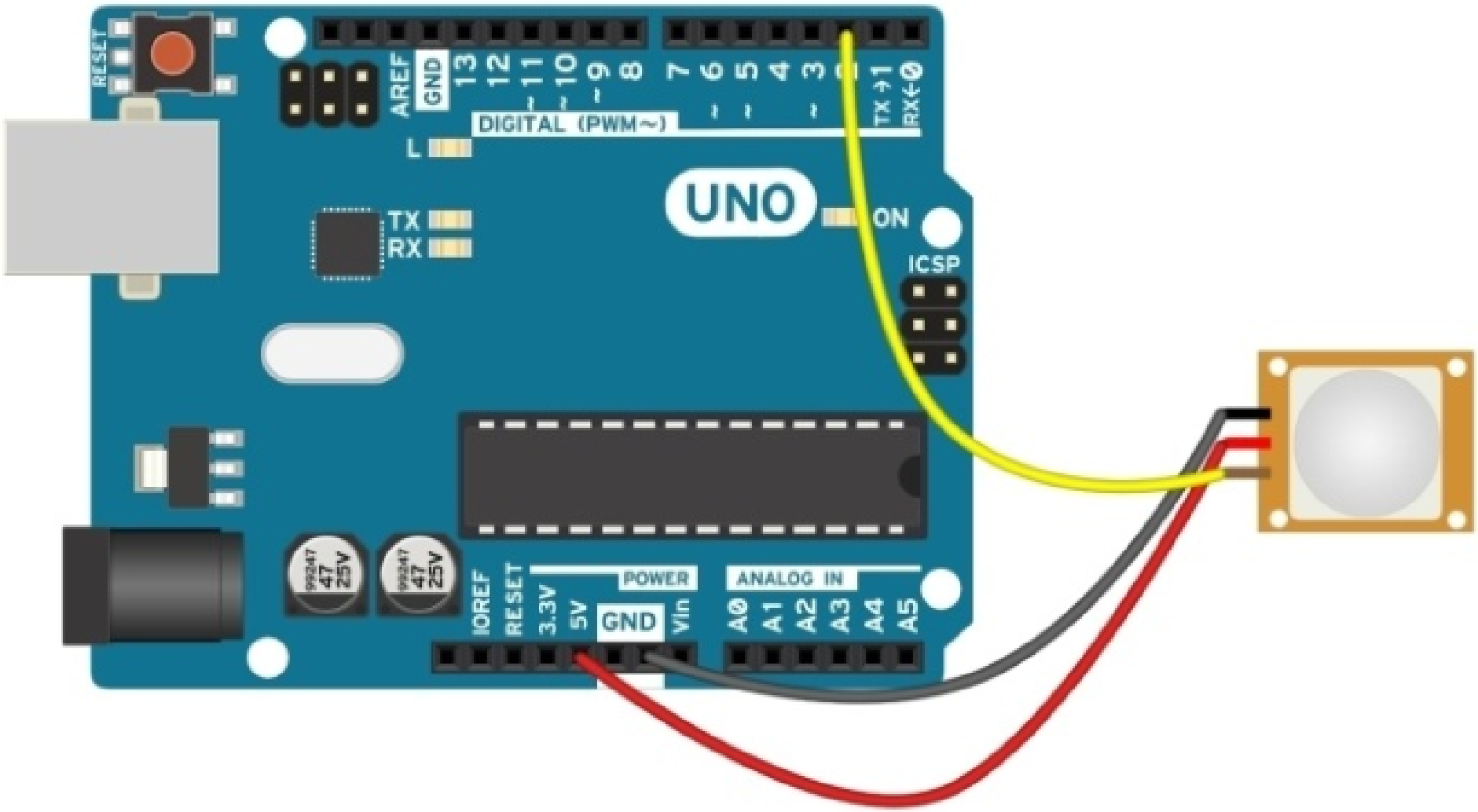
PIR Sensor

- Pyroelectric
- IR motion sensors
  - Infrared radiation
  - More the hot, more is the emission
  - Detect the motion - not the average levels

PIR Sensor Interfacing



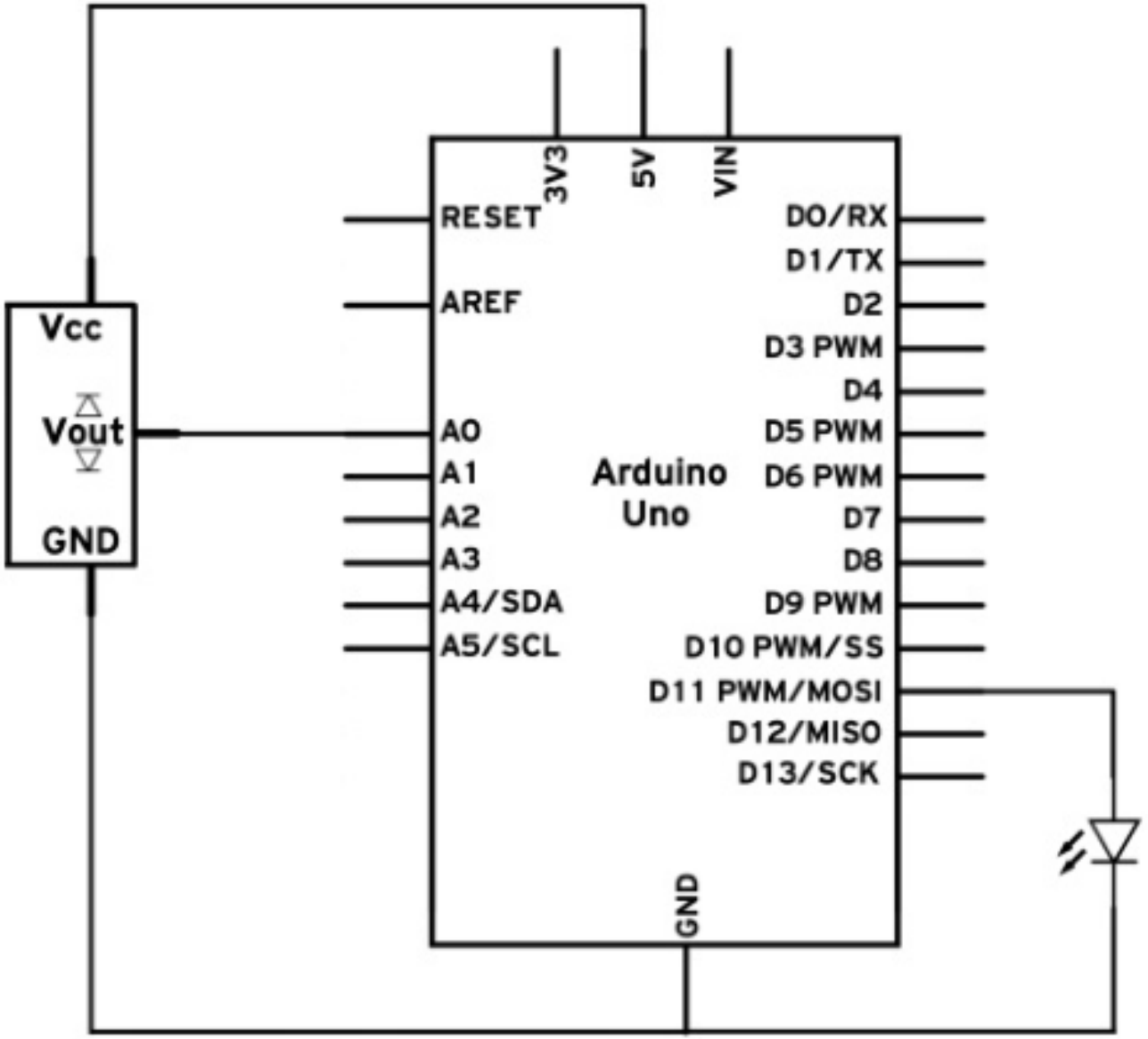
Implementation



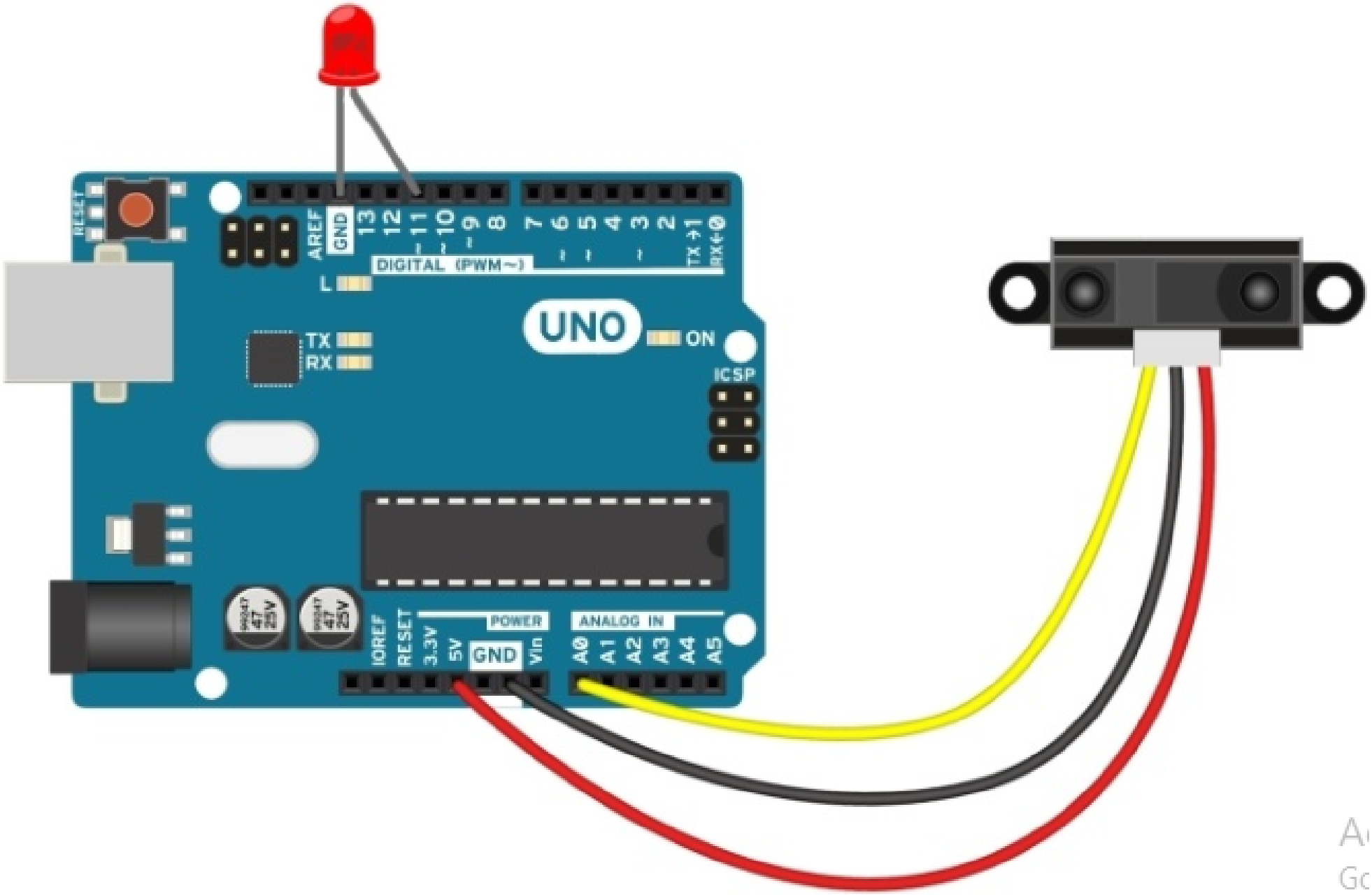
Arduino Sketch – PIR interface

- Change over to alternate document for better readability.

Distance Measurement – IR



Implementation



Arduino Sketch

- int sensorPin = A0; // Declare the used sensor pin
- int LED = 11; // Declare the connected LED
- void setup(){
- Serial.begin(9600); // Start the Serial connection
- }
- void loop(){
- // Read the analog value of the sensor
- int val = analogRead(A0);
- // Print the value over Serial
- Serial.println(val);
- // Write the value to the LED using PWM
- analogWrite(LED, val/4);
- // Wait a little for the data to print
- delay(100);
- }

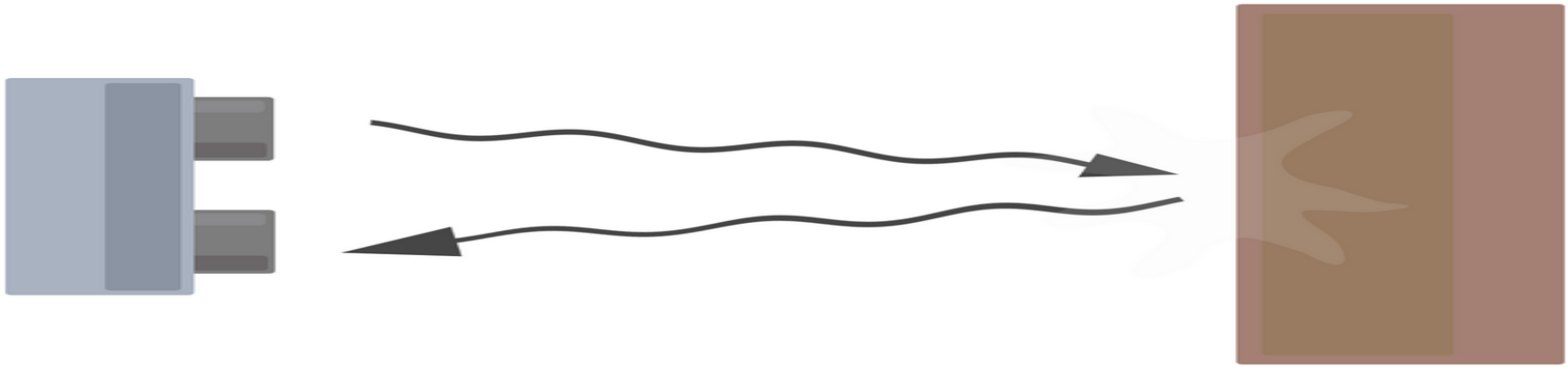
Cont...

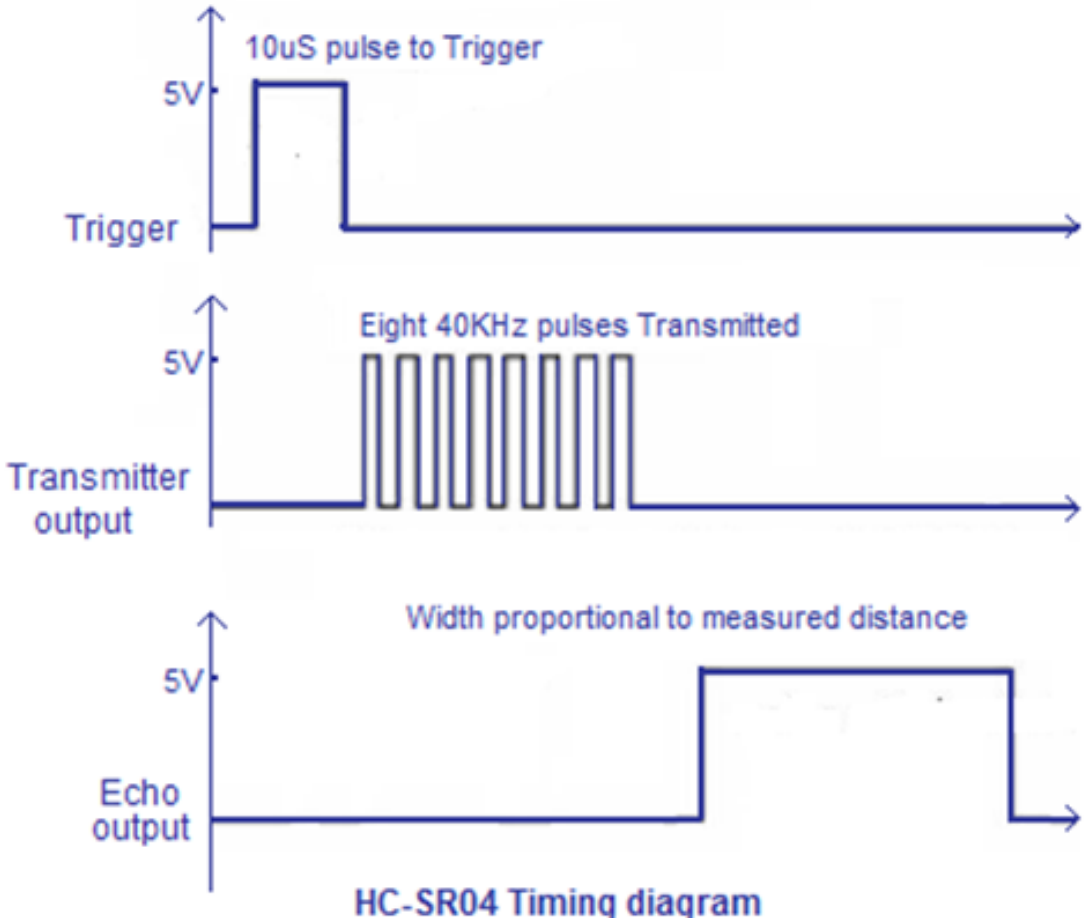
- Since PWM takes values from 0 to 255 and the analogRead() function returns values from 0 to 1023, we divide the value of analogRead() by 4 when we use it in analogWrite()

Distance Measurement - Ultrasonic

- Ultrasonic Sensor module – HC SR04 – Sends and processes the received sound signals upon a trigger.
- It measures distance by sending out a sound wave at a specific frequency and Sensing for that sound wave to bounce back.
- By recording the elapsed time between the sound wave being generated and the sound wave bouncing back, it is possible to calculate the distance between the sonar sensor and the object.

Cont...





HC-SR04 Timing diagram

HC-SR04 Pinout

Computations

- Sound speed – 340m/s
- Distance = Speed X Round trip time /2
- Calibrate distance accordingly.

Arduino Sketch

- // defines pins numbers
- const int trigPin = 9;
- const int echoPin = 10;
-
- // defines variables
- long duration;
- int distance;
-
- **void** setup() {
- pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
- pinMode(echoPin, INPUT); // Sets the echoPin as an Input
- Serial.begin(9600); // Starts the serial communication
- }

Arduino Sketch Cont...

```
void loop() {
// Clears the trigPin
digitalWrite(trigPin, LOW);
delayMicroseconds(2);

// Sets the trigPin on HIGH state for 10 micro seconds
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

// Reads the echoPin, returns the sound wave travel time in microseconds
duration = pulseIn(echoPin, HIGH);

// Calculating the distance
distance= duration*0.034/2;

// Prints the distance on the Serial Monitor
Serial.print("Distance: ");
Serial.println(distance);
}
```

Queries ?